Graphs in Machine Learning

— Spring Quarter, Lecture 3 —

Prof. Jeff Bilmes

University of Washington, Seattle
Departments of: Electrical & Computer Engineering, and Computer Science & Engineering
http://melodi.ee.washington.edu/~bilmes

April 7th, 2025

Announcements

- Reminder: in-class paper Quiz 1 will be this Wednesday, April 9th. It will consist of 25 multiple choice questions (fill in one or more bubble) and it will last 20 minutes (the first of five such in-person quizzes this quarter). It will cover everything up to and including the current lecture. Each question is all or nothing credit.
- Homework 1 is now posted, due Thursday night, April 17th at 11:59pm.
- Office hours, Wed. 10-11pm via zoom at `https://us06web.zoom.us/j/88475894076?pwd=2mFFTD93QbzFy7VIXxPFFagJN6mMga.1`

Class Road Map/Progress - EE512 - Spring 2025

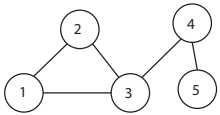
- L1 (3/31): Introduction, Overview, Graphs, Graphical Models, Graph Neural Networks
- L2 (4/2): Graphs Theory, Trees, Node and Edge Statistics, Hyper-Trees
- L3 (4/7): Node Stats, Edge Stats, Hyper-Trees, Other Graphs, Perfect Graphs
- L4 (4/9):
- L5 (4/14):
- L6 (4/16):
- L7 (4/21):
- L8 (4/23):
- L9 (4/28):
- L10 (4/30):
- L11 (5/5):
- L12 (5/7):
- L13 (5/12):
- L14 (5/14):
- L15 (5/19):
- L16 (5/21):
- LXX (5/26): Memorial day holiday
- L17 (5/28):
- L18 (6/2):
- L19 (6/4):
- Final Presentations due: (Friday 6/13/2023):

Holidays: Monday 6/26 (Memorial day). Last day of instruction: Fri, 6/7. Final Exam Week: 6/7-6/13. Our Exam Time: Fri, June 13th, 2:30 - 4:30 PM, ECE 267.

Review

More On Adjacency Matrix and Matrix Powers

- Adjacency matrix $A \in \{0,1\}^{n \times n}$ with $A[i,j] = 1$ iff $\exists$ edge between $i \in V$ and $j \in V$.
- Adjacency matrix is like a similarity, affinity, or connectedness notion, nodes are connected of they are related, or if there is a path of length one between i and j.
- Matrix power $A^k = \underbrace{AAA \dots A}_{k\times}$. Then $A^k[i,j]$ counts the number of paths there are between i and j of length k.



$$A = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix} \quad A^2 = \begin{bmatrix} 2 & 1 & 1 & 1 & 0 \\ 1 & 2 & 1 & 1 & 0 \\ 1 & 1 & 3 & 0 & 1 \\ 1 & 1 & 0 & 2 & 0 \\ 0 & 0 & 1 & 0 & 1 \end{bmatrix} \quad A^3 = \begin{bmatrix} 2 & 3 & 4 & 1 & 1 \\ 3 & 2 & 4 & 1 & 1 \\ 4 & 4 & 2 & 4 & 0 \\ 1 & 1 & 4 & 0 & 2 \\ 1 & 1 & 0 & 2 & 0 \end{bmatrix}$$

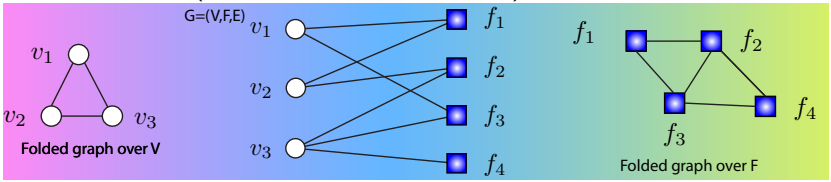
- Digraph, A is asymmetric, A^k is number of distinct directed length-k paths from i to j.
- With A sparse non-negative similarity matrix, then with $\beta < 1/\lambda_{\max}(A)$ we have

$$\lim_{k' \to \infty} \sum_{k=0}^{k'} \beta^k A^k = (I - \beta A)^{-1} \tag{2.1}$$

a form of diffusion, or (very roughly) a "geodesic" similarity, or Katz index.

Bipartite Graphs (and their folded selves)

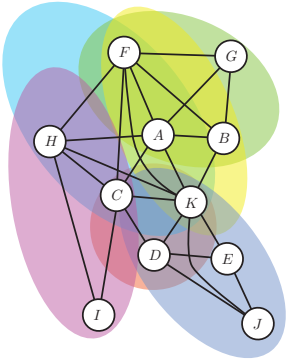
- Bipartite graph: nodes partitioned into two (disjoint) sets (V on the left, F on the right), edges exist only between (but not within any of the) sets.

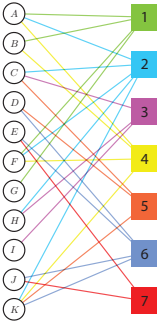


- Apps: recommendation systems (users-items, users-movies), matching problems (e.g., job applicants and positions), biological networks (e.g., genes and diseases), factor graphs.
- Any graph $G = (V, E)$ expressible by a bipartite graph $G' = (V, E, F)$ where V is on the left, G's edges E are now nodes on the right, and for each $e = \{u, v\} \in E$ we have an edge in G' from $u \in V$ to node $e \in E$ and from $v \in V$ to node $e \in E$.
- Generalizing: Multipartite graphs, nodes partitioned into (disjoint) sets $V = V_1 \cup V_2 \cup \cdots \cup V_k$ where edges can exist only between but not within sets.
- Apps: heterogeneous info. networks (e.g., authors, papers, venues), multiple-entity-type knowledge graphs, supply chain networks (products, suppliers, consumers), etc.

Drawing/Visualizing Hypergraphs & Bipartite graphs

- A set of vertices, normally edges connect two nodes.
- Hypergraph: hyperedges are shaded regions, each region a vertex cluster
- Shaded regions are cluster edge cover of "conformal" (TBD) graph.



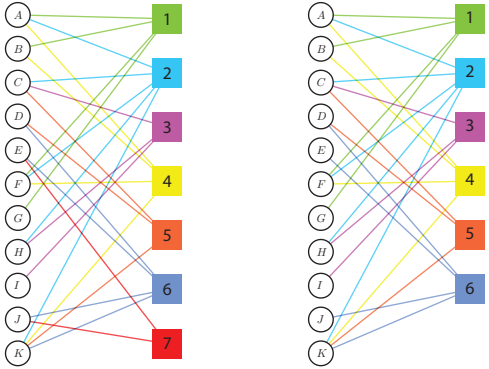


Graph of a hypergraph, conformal, and acyclic

- Let $H = (V, E)$ be a hypergraph with vertex set V and edge set E.
- The graph of a hypergraph $G(H)$ is a graph $G(H) = (V, E')$ where E' is a set of vertex pairs, and where there is an edge in E' for every pair of nodes that are in the same hyperedge.
- A hypergraph H is conformal (to the graph $G(H)$) if every clique of $G(H)$ is contained in an edge of H.
- Note, hypergraph H with hyperedges associated only with graph edges of $G(H)$ is not conformal (since cliques of $G(H)$ might not be fully contained in an edge of H).

Hypergraph vs. Reduced Hypergraph: Bipartite Form

- A hypergraph is reduced if no edge is a subset of another edge.
- Hypergraph (as bipartite graph) on left, reduced hypergraph on the right (edge {E,J} removed).



Trees defined in many ways

Theorem 2.1.3 (Trees, Berge)

Let $G = (V, E)$ be an undirected graph with $|V| = n > 2$. Then each of the following properties are equivalent and each can be used to define when G is a tree:

- *G is connected and has no cycles*
- *G has $n - 1$ edges and has no cycles,*
- *G is connected and contains exactly $n - 1$ edges,*
- *G has no cycles. Exactly one cycle created if edge added to G.*
- *G is connected, and if any edge is removed, the remaining graph is not connected,*
- *Every pair of vertices of G is connected by one unique path.*
- *G can be generated as follows: Start with v, repeatedly choose next vertex, and connect it with edge to exactly one previous vertex.*

Nodes and Degrees

- Graph $G = (V, E, w)$ where $|V| = n$ and $V = \{1, \ldots, n\}$ with adjacency matrix A:

$$A_{ij} = \begin{cases} 1 & \text{if there is an edge from node } i \text{ to node } j \\ 0 & \text{otherwise} \end{cases} \tag{2.1}$$

- If graph is weighted with $w : E \to \mathbb{R}_+$ is weight function so for $e \in E$, $w(e) \in \mathbb{R}_+$. $w(u, v) = 0$ means u is not connected to v, we have a weighted similarity, affinity, connection strength, or weighted adjacency matrix W:

$$W_{ij} = \begin{cases} w(i,j) & \text{if } (i,j) \in E \\ 0 & \text{otherwise} \end{cases}, \qquad \text{(unweighted case } w(i,j) \in \{0,1\}\text{)}. \tag{2.2}$$

  Notation A, W common in spectral graph theory, diffusion and kernel methods, and GNNs.
- degree of a vertex $v \in V$, $d_v = \deg(v) = \big|\{u \in V \mid \{u, v\} \in E\}\big|$.
- degree of a vertex in a weighted graph $d_i = \sum_{j=1}^{n} w_{ij}$, (matrix row-sum or col-sum i).
- Either case, can define degree matrix $D = \text{diag}(d_1, d_2, \ldots, d_n)$. D gives "weighted" degree (although for 0/1-valued weights, counts the number of edges incident to v_i).
- Both undirected and directed graphs, in the directed case A and W not symmetric.

In/Out Degrees

- We used "$W_{ij} = w(i,j)$ if $(i,j) \in E$" notation (i,j) deliberately. We think of an undirected graph as having edges both (i,j) and (j,i) while in directed case not necessarily so.
- (i,j) is from i to j.
- Directed graph out-degree of a vertex: $d_i^{\text{out}} = \sum_{j=1}^{n} w_{ij}$ (row sum).
- Directed graph in-degree of a vertex: $d_j^{\text{in}} = \sum_{i=1}^{n} w_{ij}$ (col-sum).

Other Core Node-Level Statistics: Eigen Centrality

- Eigenvector Centrality

$$c(v) = \frac{1}{\lambda} \sum_{u \in \mathcal{N}(v)} c(u) \quad \Rightarrow \quad \lambda c = Ac \Rightarrow \text{ Eigenvector/value problem.}$$

Node importance where connections to high-scoring neighbors contribute more. Typically computed with power iteration.

Algorithm 2: Power Iteration for Eigenvector Centrality

Input : Adjacency matrix $A \in \mathbb{R}^{n \times n}$, tolerance $\varepsilon > 0$, max iterations T
Output: Centrality vector $c \propto$ leading eigenvector of A

Initialize $c^{(0)} \leftarrow \mathbf{1}/\|\mathbf{1}\|_2$;
for $t \leftarrow 1$ **to** T **do**
  $c^{(t)} \leftarrow Ac^{(t-1)}$;
  $c^{(t)} \leftarrow c^{(t)}/\|c^{(t)}\|_2$;
  if $\|c^{(t)} - c^{(t-1)}\|_2 < \varepsilon$ **then**
    break ;
return $c^{(t)}$

Other Core Node-Level Statistics: Page Rank

- PageRank

$$c(v) = \alpha \sum_{u \in \mathcal{N}(v)} \frac{c(u)}{\deg(u)} + (1-\alpha)\frac{1}{|V|} \Rightarrow c = \alpha A D^{-1} c + (1-\alpha)\mathbf{1}/n$$

random-walk-based importance (similar to Eigen centrality) but with restart probability $1-\alpha$, where D is diagonal matrix $D_{ii} = \deg(i)$, A adjacency matrix (directed/undirected).

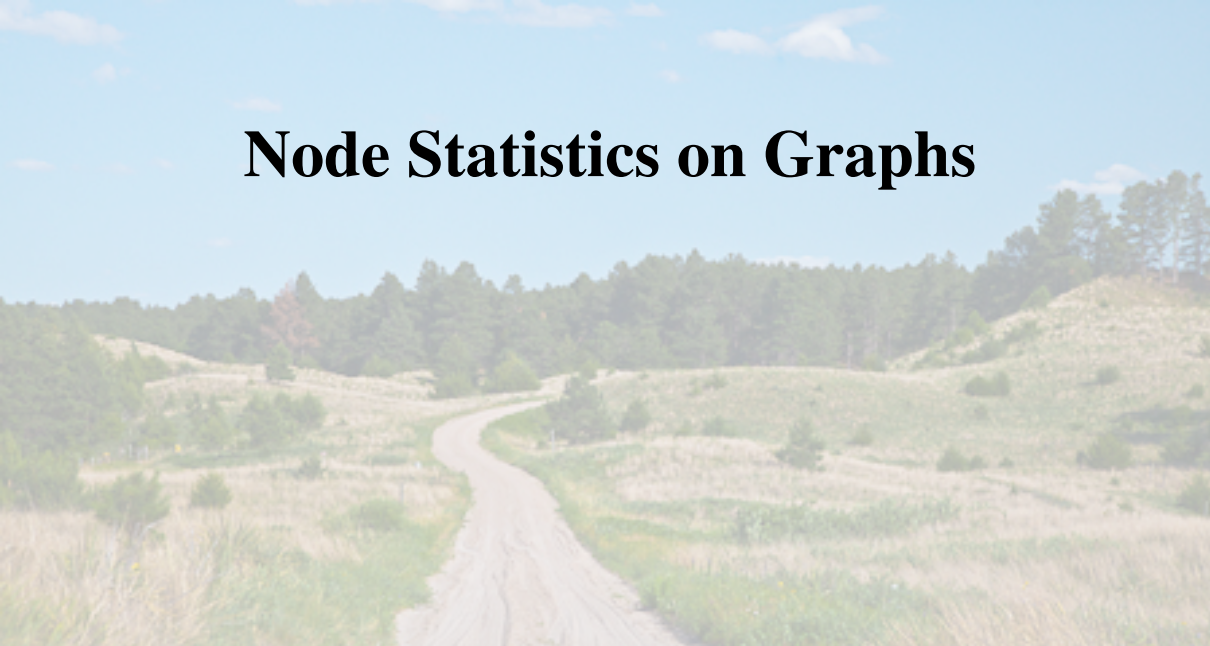
- Again, Eigen equation solvable with power iteration:

Algorithm 3: An Example of Power Iteration for a PageRank Variant

Input : Adjacency matrix $A \in \mathbb{R}^{n \times n}$, damping factor $\alpha \in (0,1)$, tolerance ε, max iterations T
Output: PageRank vector $c \in \mathbb{R}^n$

Compute degree vector $d_i \leftarrow \sum_j A_{ij}$;
Initialize $c^{(0)} \leftarrow \mathbf{1}/n$ **for** $t \leftarrow 1$ **to** T **do**
  for $i \leftarrow 1$ **to** n **do**
    $c_i^{(t)} \leftarrow \alpha \sum_{j=1}^{n} \frac{A_{ji}}{d_j} c_j^{(t-1)} + (1-\alpha)/n$
  if $\|c^{(t)} - c^{(t-1)}\|_1 < \varepsilon$ **then**
    break
return $c^{(t)}$

Node Statistics on Graphs



Perron–Frobenius theorem, positive or non-negative matrices and centrality

Theorem 3.2.1 (Perron–Frobenius)

Let A be an $n \times n$ matrix, all entries strictly positive, i.e. $a_{ij} > 0$ for $1 \leq i, j \leq n$. Then:

1. *Existence and Uniqueness of a Dominant Eigenvalue: There is a positive real number r, called the Perron root or leading eigenvalue, such that r is an eigenvalue of A, and any other eigenvalue λ (possibly complex) satisfies $|\lambda| < r$. In particular, the spectral radius $\rho(A)$ of A equals r.*
2. *Existence of a Positive Eigenvector: $\exists$ an eigenvector $v = (v_1, \ldots, v_n)^T$ corresponding to r such that $v_i > 0$ for all $1 \leq i \leq n$, i.e. $A\,v = rv$. Similarly, there is a positive left eigenvector w such that $w^T A = r\,w^T$.*

- For strictly positive matrices, r (the Perron root) dominates the spectrum; its eigenvalue is real and larger in magnitude than any other eigenvalue.
- Re Eigen centrality & PageRank: Both rely on finding a principal eigenvector of a nonnegative matrix. A *maximal* eigenvalue with a strictly positive eigenvector still exists, thus these centrality measures converge to provide meaningful rankings.

Vertex and Edge Induced Subgraphs & Induced Degree

- Given a graph $G = (V, E)$, it is possible to construct either a vertex- or an edge-induced subgraph.

Vertex and Edge Induced Subgraphs & Induced Degree

- Given a graph $G = (V, E)$, it is possible to construct either a vertex- or an edge-induced subgraph.
- Given subset $S \subseteq V$, then $G' = (S, E(S))$ is a vertex induced subgraph, $E(S) = E \cap (S \times S)$.

Vertex and Edge Induced Subgraphs & Induced Degree

- Given a graph $G = (V, E)$, it is possible to construct either a vertex- or an edge-induced subgraph.
- Given subset $S \subseteq V$, then $G' = (S, E(S))$ is a vertex induced subgraph, $E(S) = E \cap (S \times S)$.
- Given subset $\tilde{E} \subseteq E$, then $G(\tilde{E}) = (V(\tilde{E}), \tilde{E})$ is edge-induced subgraph, $V(\tilde{E}) = V \cap \left\{ u \in V : \exists v \text{ s.t. } (u, v) \in \tilde{E} \right\}$.

Vertex and Edge Induced Subgraphs & Induced Degree

- Given a graph $G = (V, E)$, it is possible to construct either a vertex- or an edge-induced subgraph.
- Given subset $S \subseteq V$, then $G' = (S, E(S))$ is a vertex induced subgraph, $E(S) = E \cap (S \times S)$.
- Given subset $\tilde{E} \subseteq E$, then $G(\tilde{E}) = (V(\tilde{E}), \tilde{E})$ is edge-induced subgraph, $V(\tilde{E}) = V \cap \left\{ u \in V : \exists v \text{ s.t. } (u, v) \in \tilde{E} \right\}$.
- Usually, "induced subgraph" implies "vertex induced subgraph" when it is not specified. Also, we contrast induced subgraphs with spanning subgraphs defined later below.

Vertex and Edge Induced Subgraphs & Induced Degree

- Given a graph $G = (V, E)$, it is possible to construct either a vertex- or an edge-induced subgraph.
- Given subset $S \subseteq V$, then $G' = (S, E(S))$ is a vertex induced subgraph, $E(S) = E \cap (S \times S)$.
- Given subset $\tilde{E} \subseteq E$, then $G(\tilde{E}) = (V(\tilde{E}), \tilde{E})$ is edge-induced subgraph, $V(\tilde{E}) = V \cap \left\{ u \in V : \exists v \text{ s.t. } (u, v) \in \tilde{E} \right\}$.
- Usually, "induced subgraph" implies "vertex induced subgraph" when it is not specified. Also, we contrast induced subgraphs with spanning subgraphs defined later below.
- Define the degree of s in the subgraph as $d_s(\tilde{E}) = |\delta_s(\tilde{E})|$ where $\delta_s(\tilde{E}) = \left\{ t \in V | (s, t) \in \tilde{E} \right\}$ is the set of neighbors of s in $G(\tilde{E})$.

Vertex and Edge Induced Subgraphs & Induced Degree

- Given a graph $G = (V, E)$, it is possible to construct either a vertex- or an edge-induced subgraph.
- Given subset $S \subseteq V$, then $G' = (S, E(S))$ is a vertex induced subgraph, $E(S) = E \cap (S \times S)$.
- Given subset $\tilde{E} \subseteq E$, then $G(\tilde{E}) = (V(\tilde{E}), \tilde{E})$ is edge-induced subgraph, $V(\tilde{E}) = V \cap \left\{ u \in V : \exists v \text{ s.t. } (u, v) \in \tilde{E} \right\}$.
- Usually, "induced subgraph" implies "vertex induced subgraph" when it is not specified. Also, we contrast induced subgraphs with spanning subgraphs defined later below.
- Define the degree of s in the subgraph as $d_s(\tilde{E}) = |\delta_s(\tilde{E})|$ where $\delta_s(\tilde{E}) = \left\{ t \in V | (s, t) \in \tilde{E} \right\}$ is the set of neighbors of s in $G(\tilde{E})$.
- Weighted version: $d_s(\tilde{E}) = \sum_{t \in V : (s,t) \in \tilde{E}} w_{s,t}$

Graph Laplacian

- An amazing amount of information about a graph can be expressed by properties of its Laplacian matrix.

Graph Laplacian

- An amazing amount of information about a graph can be expressed by properties of its Laplacian matrix.
- Given degree matrix D (i.e., a diagonal matrix with entries $d_i = \sum_{j=1}^{n} w_{ij}$) and symmetric adjacency matrix W, define the unnormalized graph Laplacian matrix as

$$L = D - W \tag{3.1}$$

Graph Laplacian

- An amazing amount of information about a graph can be expressed by properties of its Laplacian matrix.
- Given degree matrix D (i.e., a diagonal matrix with entries $d_i = \sum_{j=1}^{n} w_{ij}$) and symmetric adjacency matrix W, define the unnormalized graph Laplacian matrix as

$$L = D - W \tag{3.1}$$

- Note $L_{ii} = d_{ii} - w_{ii} = \sum_{j=1}^{n} w_{ij} - w_{ii} = \sum_{j \neq i} w_{ij}$, so diagonal of W is irrelevant to L (e.g., self-edges & self-similarities don't matter).

Properties of the graph Laplacian

Proposition 3.2.2 (Properties of the unnormalized graph Laplacian L)

1. *For every vector $z \in \mathbb{R}^n$, we have, for $L = D - W$,*

$$z^\intercal L z = \frac{1}{2} \sum_{i,j=1}^{n} w_{ij}(z_i - z_j)^2 \geq 0 \tag{3.2}$$

2. *L is symmetric and positive semi-definite.*
3. *The smallest eigenvalue of L is 0, and the corresponding eigenvector is the constant one vector $\mathbb{1} = \mathbb{1}_V \in \mathbb{R}^n$.*
4. *L has n non-negative real valued eigenvalues $0 = \lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_n$.*

Properties of the graph Laplacian

Proposition 3.2.2 (Properties of the unnormalized graph Laplacian L)

1. *For every vector $z \in \mathbb{R}^n$, we have, for $L = D - W$,*
$$z^\intercal L z = \frac{1}{2} \sum_{i,j=1}^{n} w_{ij}(z_i - z_j)^2 \geq 0 \tag{3.2}$$
2. *L is symmetric and positive semi-definite.*
3. *The smallest eigenvalue of L is 0, and the corresponding eigenvector is the constant one vector $\mathbb{1} = \mathbb{1}_V \in \mathbb{R}^n$.*
4. *L has n non-negative real valued eigenvalues $0 = \lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_n$.*

$$z^\intercal L z = z^\intercal D z - z^\intercal W z = \sum_{i=1}^{n} d_i z_i^2 - \sum_{i,j=1}^{n} z_i z_j w_{ij} \tag{3.3}$$

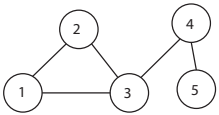
$$= \frac{1}{2}\left(\sum_{i=1}^{n} d_i z_i^2 - 2 \sum_{i,j=1}^{n} z_i z_j w_{ij} + \sum_{j=1}^{n} d_j z_j^2 \right) = \frac{1}{2} \sum_{ij=1}^{n} w_{ij}(z_i - z_j)^2$$

Recall from Lecture 2

- The next slide is a repeat from lecture 2.

More On Adjacency Matrix and Matrix Powers

- Adjacency matrix $A \in \{0,1\}^{n \times n}$ with $A[i,j] = 1$ iff $\exists$ edge between $i \in V$ and $j \in V$.
- Adjacency matrix is like a similarity, affinity, or connectedness notion, nodes are connected of they are related, or if there is a path of length one between i and j.
- Matrix power $A^k = \underbrace{AAA \dots A}_{k\times}$. Then $A^k[i,j]$ counts the number of paths there are between i and j of length k.



$$A = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix} \quad A^2 = \begin{bmatrix} 2 & 1 & 1 & 1 & 0 \\ 1 & 2 & 1 & 1 & 0 \\ 1 & 1 & 3 & 0 & 1 \\ 1 & 1 & 0 & 2 & 0 \\ 0 & 0 & 1 & 0 & 1 \end{bmatrix} \quad A^3 = \begin{bmatrix} 2 & 3 & 4 & 1 & 1 \\ 3 & 2 & 4 & 1 & 1 \\ 4 & 4 & 2 & 4 & 0 \\ 1 & 1 & 4 & 0 & 2 \\ 1 & 1 & 0 & 2 & 0 \end{bmatrix}$$

- Digraph, A is asymmetric, A^k is number of distinct directed length-k paths from i to j.
- With A sparse non-negative similarity matrix, then with $\beta < 1/\lambda_{\max}(A)$ we have

$$\lim_{k' \to \infty} \sum_{k=0}^{k'} \beta^k A^k = (I - \beta A)^{-1} \tag{3.1}$$

a form of diffusion, or (very roughly) a "geodesic" similarity, or Katz index.

Diffusion Kernel Based on Laplacian

- Recall A^k is number of distinct length-k paths from i to j, and diffusion, i.e., A sparse non-negative similarity matrix, then with $\beta < 1/\lambda_{\max}(A)$ we have "geodesic" variant:

$$\lim_{k\to\infty} \sum_k \beta^k A^k = (I - \beta A)^{-1} = \lim_{k\to\infty} \sum_k X^k = (I - X)^{-1} \tag{3.4}$$

  a "geodesic" or "transitive" similarity or relationship strength.

Diffusion Kernel Based on Laplacian

- Recall A^k is number of distinct length-k paths from i to j, and diffusion, i.e., A sparse non-negative similarity matrix, then with $\beta < 1/\lambda_{\max}(A)$ we have "geodesic" variant:

$$\lim_{k\to\infty} \sum_k \beta^k A^k = (I - \beta A)^{-1} = \lim_{k\to\infty} \sum_k X^k = (I - X)^{-1} \tag{3.4}$$

  a "geodesic" or "transitive" similarity or relationship strength.
- Consider a graph Laplacian form of this (a diffusion kernel) with $\lambda > 0$

$$K = (D - W + \lambda I)^{-1} = (L + \lambda I)^{-1} \tag{3.5}$$

  This is the inverse of symmetric, positive-definite matrix w. non-positive off-diagonals (P.D. M-matrix), can be shown to have non-negative entries https://en.wikipedia.org/wiki/M-matrix

Diffusion Kernel Based on Laplacian

- Recall A^k is number of distinct length-k paths from i to j, and diffusion, i.e., A sparse non-negative similarity matrix, then with $\beta < 1/\lambda_{\max}(A)$ we have "geodesic" variant:

$$\lim_{k\to\infty} \sum_k \beta^k A^k = (I - \beta A)^{-1} = \lim_{k\to\infty} \sum_k X^k = (I - X)^{-1} \tag{3.4}$$

  a "geodesic" or "transitive" similarity or relationship strength.
- Consider a graph Laplacian form of this (a diffusion kernel) with $\lambda > 0$

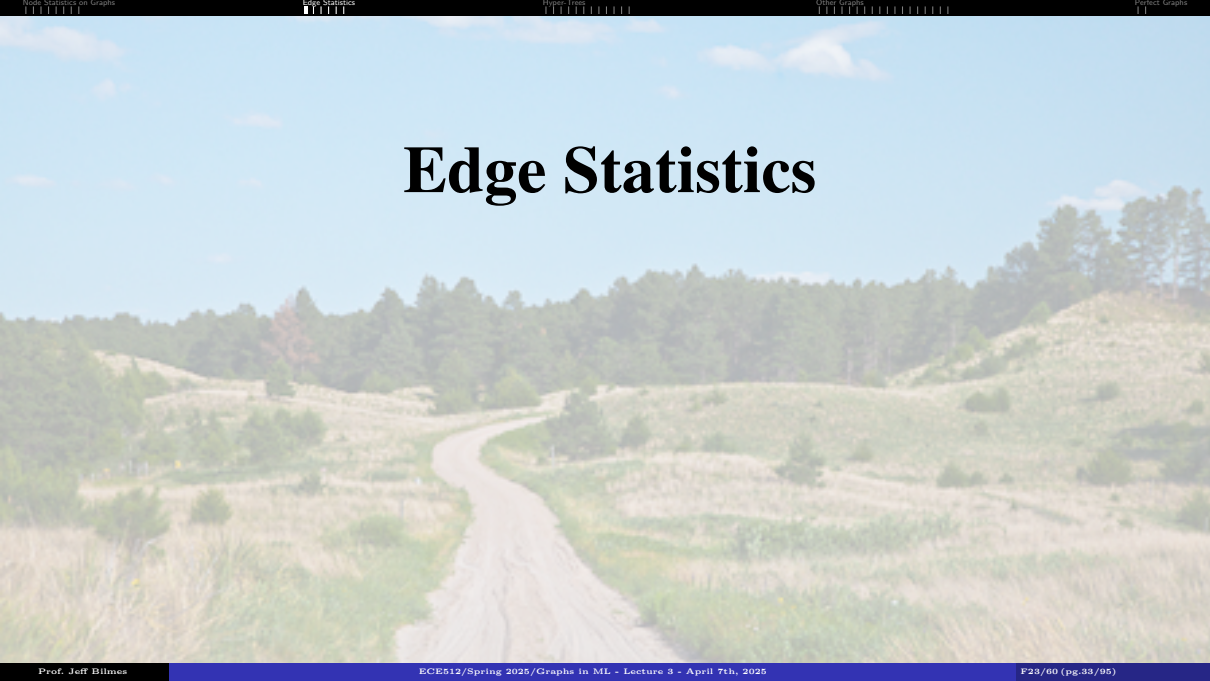
$$K = (D - W + \lambda I)^{-1} = (L + \lambda I)^{-1} \tag{3.5}$$

  This is the inverse of symmetric, positive-definite matrix w. non-positive off-diagonals (P.D. M-matrix), can be shown to have non-negative entries https://en.wikipedia.org/wiki/M-matrix
- We can equate the above,by setting $(I - X) = L + \lambda I$, to get:

$$X = (1 - \lambda)I - L = W - D + (1 - \lambda)I$$

  - W: weighted adjacency (direct affinities), off-diagonal entries same as W.
  - $-D$: discounts/penalizes high-degree nodes (hub suppression, deflation, penalize popularity)
  - $(1 - \lambda)I$: regularization, identity bias, controls self-weight, further deflation, as λ increases decays more over path length.

Edge Statistics



Edge Stats

- There exist edge-based statistics too, useful for thinks like link prediction, edge classification, and/or edge weighting. Here are a few important ones:

Edge Stats

- There exist edge-based statistics too, useful for thinks like link prediction, edge classification, and/or edge weighting. Here are a few important ones:
- Neighborhood Overlap,useful in link prediction.

$$S[i,j] = |\mathcal{N}(i) \cap \mathcal{N}(j)| \tag{3.6}$$

Edge Stats

- There exist edge-based statistics too, useful for thinks like link prediction, edge classification, and/or edge weighting. Here are a few important ones:
- Neighborhood Overlap,useful in link prediction.

$$S[i,j] = |\mathcal{N}(i) \cap \mathcal{N}(j)| \tag{3.6}$$

- Jaccard similarity, normalized neighborhood overlap:

$$J[i,j] = \frac{|\mathcal{N}(i) \cap \mathcal{N}(j)|}{|\mathcal{N}(i) \cup \mathcal{N}(j)|} \tag{3.7}$$

Edge Stats

- There exist edge-based statistics too, useful for thinks like link prediction, edge classification, and/or edge weighting. Here are a few important ones:
- Neighborhood Overlap,useful in link prediction.

$$S[i,j] = |\mathcal{N}(i) \cap \mathcal{N}(j)| \tag{3.6}$$

- Jaccard similarity, normalized neighborhood overlap:

$$J[i,j] = \frac{|\mathcal{N}(i) \cap \mathcal{N}(j)|}{|\mathcal{N}(i) \cup \mathcal{N}(j)|} \tag{3.7}$$

- Adamic-Adar index, common neighbors weighted inversely by their popularity. E.g., in link prediction to suggest new friends based on shared **rare** connections.

$$\text{AA}[i,j] = \sum_{z \in \mathcal{N}(i) \cap \mathcal{N}(j)} \frac{1}{\log |\mathcal{N}(z)|} \tag{3.8}$$

Edge Stats

- There exist edge-based statistics too, useful for thinks like link prediction, edge classification, and/or edge weighting. Here are a few important ones:
- Neighborhood Overlap,useful in link prediction.

$$S[i,j] = |\mathcal{N}(i) \cap \mathcal{N}(j)| \tag{3.6}$$

- Jaccard similarity, normalized neighborhood overlap:

$$J[i,j] = \frac{|\mathcal{N}(i) \cap \mathcal{N}(j)|}{|\mathcal{N}(i) \cup \mathcal{N}(j)|} \tag{3.7}$$

- Adamic-Adar index, common neighbors weighted inversely by their popularity. E.g., in link prediction to suggest new friends based on shared **rare** connections.

$$\text{AA}[i,j] = \sum_{z \in \mathcal{N}(i) \cap \mathcal{N}(j)} \frac{1}{\log |\mathcal{N}(z)|} \tag{3.8}$$

- Preferential Attachment, aspects of "rich get richer" in networks, e.g., in link prediction, to estimate the likelihood of new edges forming between high-degree nodes:

$$\text{PA}[i,j] = \deg(i) \cdot \deg(j) \tag{3.9}$$

Edge Stats

- There exist edge-based statistics too, useful for thinks like link prediction, edge classification, and/or edge weighting. Here are a few important ones:
- Neighborhood Overlap,useful in link prediction.

$$S[i,j] = |\mathcal{N}(i) \cap \mathcal{N}(j)| \tag{3.6}$$

- Jaccard similarity, normalized neighborhood overlap:

$$J[i,j] = \frac{|\mathcal{N}(i) \cap \mathcal{N}(j)|}{|\mathcal{N}(i) \cup \mathcal{N}(j)|} \tag{3.7}$$

- Adamic-Adar index, common neighbors weighted inversely by their popularity. E.g., in link prediction to suggest new friends based on shared **rare** connections.

$$\text{AA}[i,j] = \sum_{z \in \mathcal{N}(i) \cap \mathcal{N}(j)} \frac{1}{\log |\mathcal{N}(z)|} \tag{3.8}$$

- Preferential Attachment, aspects of "rich get richer" in networks, e.g., in link prediction, to estimate the likelihood of new edges forming between high-degree nodes:

$$\text{PA}[i,j] = \deg(i) \cdot \deg(j) \tag{3.9}$$

- Edge weight itself viewable a statistic (e.g., distance/similarity between node reps.).

From Lecture 2

- Recall next slide from lecture 2.

Still Other Core Node-Level Statistics

- Local (Node) Clustering Coefficient:

$$c_v = \frac{2 \cdot T(v)}{\deg(v)(\deg(v)-1)} = \frac{T(v)}{\binom{\deg(v)}{2}}$$

  Proportion of a node's neighbors that are connected to each other; $T(v)$ is the number of triangles that include v, i.e., $T(v) = \left|\left\{\{u,w\} \in E : u,w \in \mathcal{N}(v)\right\}\right|$, and again $\mathcal{N}(v)$ are v's neighbors.
- K-Core Number: The largest k such that node v is in the k-core (a maximal v-containing subgraph where all nodes have degree $\geq k$), a form of "buried" centrality.
- Eccentricity:

$$\varepsilon(v) = \max_{u \in V} d(v,u)$$

  Distance from v to the farthest reachable node.
- Graphlets, a vector of counts of rooted subgraphs at a given node up to a given size (generalizing both degree (size 2) and number of triangles (size 3) a node is involved in).
- There are many others ...

Structural/Topological graph statistics

- Edge betweenness centrality

$$\mathrm{EB}(i,j) = \sum_{s \neq t} \frac{\sigma_{st}(i,j)}{\sigma_{st}} \tag{3.10}$$

Number of shortest paths passing through edge (i,j).

Structural/Topological graph statistics

- Edge betweenness centrality

$$\text{EB}(i,j) = \sum_{s \neq t} \frac{\sigma_{st}(i,j)}{\sigma_{st}} \tag{3.10}$$

  Number of shortest paths passing through edge (i,j).

- Edge clustering coefficient: Measures how well-connected the endpoints' neighbors are, or how many triples are triangles:

$$\text{ECC}(i,j) = \frac{|\text{triangles including } (i,j)|}{\min(\deg(i)-1, \deg(j)-1)} \tag{3.11}$$

Structural/Topological graph statistics

- Edge betweenness centrality

$$\text{EB}(i,j) = \sum_{s \neq t} \frac{\sigma_{st}(i,j)}{\sigma_{st}} \tag{3.10}$$

  Number of shortest paths passing through edge (i,j).

- Edge clustering coefficient: Measures how well-connected the endpoints' neighbors are, or how many triples are triangles:

$$\text{ECC}(i,j) = \frac{|\text{triangles including } (i,j)|}{\min(\deg(i)-1, \deg(j)-1)} \tag{3.11}$$

- Edge embeddedness: Same as common neighbors: how embedded is the edge in shared structure.

Weighted or attribute-based variants

- When nodes or edges have features then we can measure feature distance or similarity between node i and j via the nodes' features (using Euclidean distance, cosine similarity, etc.).
- The edge weight itself can be treated as a statistic.
- Temporal difference in dynamic graphs also a useful statistic (temporal locality).
- As mentioned above, node, edge, and graph statistics have been used in many applications including:
  - Link prediction – predict missing or future edges
  - Edge classification/regression – predicting edge type or a weight for an edge.
  - Edge sampling sparsification – to increase graph sparsity while maintaining representativeness.

Hyper Trees

Tree defs Lecture 2

- Recall the next slide from lecture 2 on the many definitions of a tree.

Trees defined in many ways

Theorem 3.4.3 (Trees, Berge)

Let $G = (V, E)$ be an undirected graph with $|V| = n > 2$. Then each of the following properties are equivalent and each can be used to define when G is a tree:

- *G is connected and has no cycles*
- *G has $n - 1$ edges and has no cycles,*
- *G is connected and contains exactly $n - 1$ edges,*
- *G has no cycles. Exactly one cycle created if edge added to G.*
- *G is connected, and if any edge is removed, the remaining graph is not connected,*
- *Every pair of vertices of G is connected by one unique path.*
- *G can be generated as follows: Start with v, repeatedly choose next vertex, and connect it with edge to exactly one previous vertex.*

k-trees

Generalizations of a tree as defined as follows:

Definition 3.4.1 (k-tree)

A complete graph with $k+1$ nodes is a k-tree. To construct a k tree with $n+1$ nodes starting from a k-tree with n nodes, choose some size k complete sub-graph of the n-node k-tree and connect the $n+1$'st node to all nodes in the k-node complete sub-graph.

k-trees

Generalizations of a tree as defined as follows:

Definition 3.4.1 (k-tree)

A complete graph with $k+1$ nodes is a k-tree. To construct a k tree with $n+1$ nodes starting from a k-tree with n nodes, choose some size k complete sub-graph of the n-node k-tree and connect the $n+1$'st node to all nodes in the k-node complete sub-graph.

- Any complete n-graph is an $n-1$-tree

k-trees

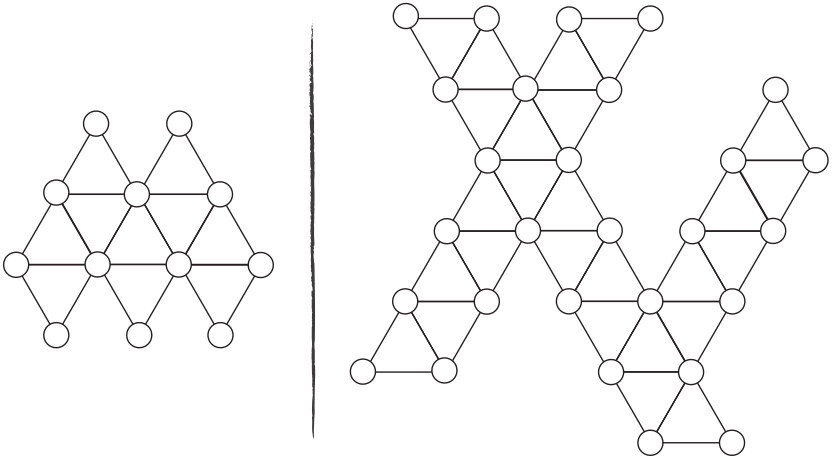
Generalizations of a tree as defined as follows:

Definition 3.4.1 (k-tree)

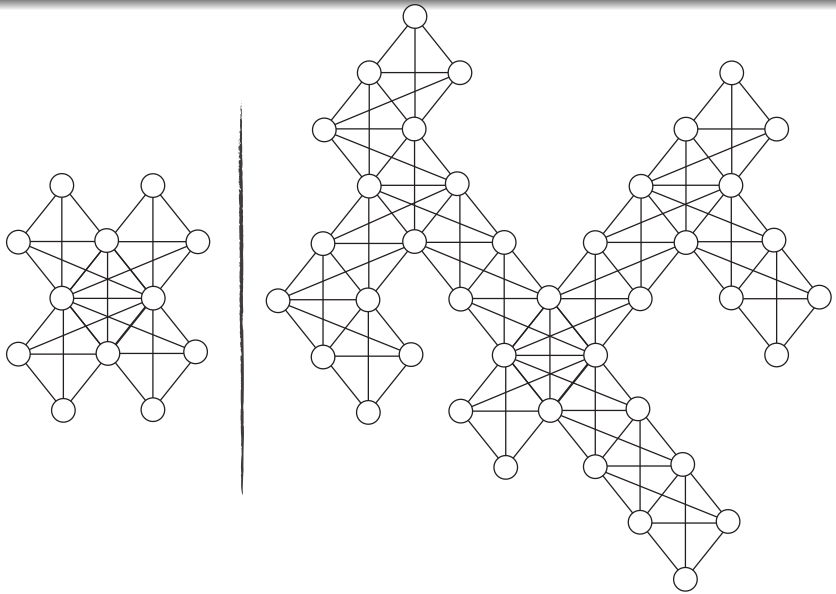
A complete graph with $k+1$ nodes is a k-tree. To construct a k tree with $n+1$ nodes starting from a k-tree with n nodes, choose some size k complete sub-graph of the n-node k-tree and connect the $n+1$'st node to all nodes in the k-node complete sub-graph.

- Any complete n-graph is an $n-1$-tree
- a regular tree is a 1-tree.

Example of 2-trees



Example of 3-trees



Graph separators

- Given $a, b \in V$, $a \neq b$, a set $S \subseteq V$ is an (a, b)-**separator** in G, if all paths from a to b must intersect some node in S.

Graph separators

- Given $a, b \in V$, $a \neq b$, a set $S \subseteq V$ is an (a, b)-**separator** in G, if all paths from a to b must intersect some node in S.
- A **minimal** (a, b)-**separator** S is an (a, b)-separator such that any strict subset $S' \subset S$ is no longer an (a, b)-separator.

Graph separators

- Given $a, b \in V$, $a \neq b$, a set $S \subseteq V$ is an (a, b)-**separator** in G, if all paths from a to b must intersect some node in S.
- A **minimal** (a, b)-**separator** S is an (a, b)-separator such that any strict subset $S' \subset S$ is no longer an (a, b)-separator.
- A set S is a **separator** in $G = (V, E)$ if there exists $a, b \in V$ such that S is an (a, b)-separator.

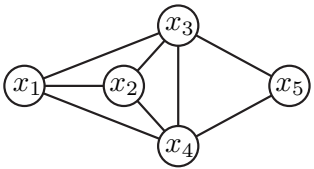
Graph separators

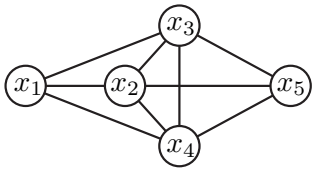
- Given $a, b \in V$, $a \neq b$, a set $S \subseteq V$ is an (a, b)-**separator** in G, if all paths from a to b must intersect some node in S.
- A **minimal** (a, b)-**separator** S is an (a, b)-separator such that any strict subset $S' \subset S$ is no longer an (a, b)-separator.
- A set S is a **separator** in $G = (V, E)$ if there exists $a, b \in V$ such that S is an (a, b)-separator.
- a set S is a **minimal separator** if there exists an $a, b \in V$ such that S is a minimal (a, b)-separator.

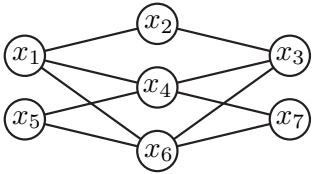
Graph separators

- Given $a, b \in V$, $a \neq b$, a set $S \subseteq V$ is an (a, b)-**separator** in G, if all paths from a to b must intersect some node in S.
- A **minimal** (a, b)-**separator** S is an (a, b)-separator such that any strict subset $S' \subset S$ is no longer an (a, b)-separator.
- A set S is a **separator** in $G = (V, E)$ if there exists $a, b \in V$ such that S is an (a, b)-separator.
- a set S is a **minimal separator** if there exists an $a, b \in V$ such that S is a minimal (a, b)-separator.
- Alternate definition of minimal separator is if S shatters graph into (i.e., $G[V \setminus S]$ is) two or more connected components and no strict subset of S does so in the same way.

Graph separators - examples

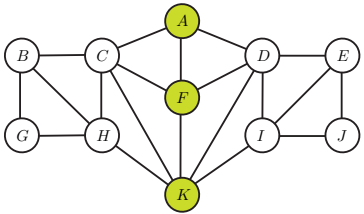


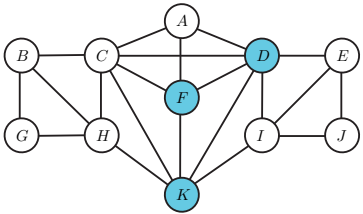


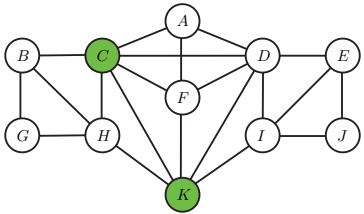


- Left: both $\{x_3, x_4\}$ and $\{x_2, x_3, x_4\}$ a (x_1, x_5)-separator, only $\{x_3, x_4\}$ is a minimal (x_1, x_5)-separator.
- Middle: $\{x_3, x_4\}$ no longer a separator. $\{x_2, x_3, x_4\}$ now a minimal (x_1, x_5)-separator.
- Right: $\{x_2, x_4, x_6\}$ minimal (x_1, x_3)-separator, $\{x_4, x_6\}$ is a minimal (x_5, x_7)-separator.

Graph separators - examples







- Left: A, F, K is a minimal (B, E)-separator.
- Middle: D, F, K is a non-minimal (B, E)-separator
- Right: C, K is a minimal (B, E)-separator

Minimal separators - property

Lemma 3.4.2

Let S be a minimal (a, b)-separator in $G = (V, E)$ and let G_A, G_B be the connected components of G once S is removed containing a and b (i.e., $(G_A, G_B) \subseteq G[V \setminus S]$) where $a \in V(G_A)$ and $b \in V(G_B)$. Then each $s \in S$ is adjacent both to some node in G_A and some node in G_B.

Minimal separators - property

Lemma 3.4.2

Let S be a minimal (a, b)-separator in $G = (V, E)$ and let G_A, G_B be the connected components of G once S is removed containing a and b (i.e., $(G_A, G_B) \subseteq G[V \setminus S]$) where $a \in V(G_A)$ and $b \in V(G_B)$. Then each $s \in S$ is adjacent both to some node in G_A and some node in G_B.

Proof.

Suppose the contrary, that there exists an $s \in S$ not adjacent to any $v \in G_A$. In such case, $S \setminus \{s\}$ is still an (a, b)-separator since no path from G_A can get directly through s, contradicting the minimality of S. □

k-trees

- In a tree, all minimal separators are size 1

k-trees

- In a tree, all minimal separators are size 1
- In a k-tree, all minimal separators are size k (and thus a k-clique).

k-trees

- In a tree, all minimal separators are size 1
- In a k-tree, all minimal separators are size k (and thus a k-clique).
- In a k-tree, all maxcliques are size $k+1$, so maximum clique size is $k+1$.

k-trees

- In a tree, all minimal separators are size 1
- In a k-tree, all minimal separators are size k (and thus a k-clique).
- In a k-tree, all maxcliques are size $k+1$, so maximum clique size is $k+1$.
- even stronger:

k-trees

- In a tree, all minimal separators are size 1
- In a k-tree, all minimal separators are size k (and thus a k-clique).
- In a k-tree, all maxcliques are size $k+1$, so maximum clique size is $k+1$.
- even stronger:

Lemma 3.4.3

A graph $G = (V, E)$ is a k-tree iff

- *G is connected*
- *G's maximum clique is of size $k+1$*
- *Every minimal separator of G is a k-clique.*

Spanning Sub-graphs

Definition 3.4.4 (Spanning Sub-graph)

A graph $G' = (V, E')$ is a spanning sub-graph of another graph $G = (V, E)$ if $E' \subseteq E$. I.e., G' is a spanning subgraph of G if G' can be embedded into G, specifically G' has the same node set as G and a subset of its edges: $E' \subseteq E$.

- Hence, a spanning tree of a graph $G = (V, E)$ is tree $T = (V, E')$ with the same node set V but E' are the edges of a tree.

Spanning Sub-graphs

Definition 3.4.4 (Spanning Sub-graph)

A graph $G' = (V, E')$ is a spanning sub-graph of another graph $G = (V, E)$ if $E' \subseteq E$. I.e., G' is a spanning subgraph of G if G' can be embedded into G, specifically G' has the same node set as G and a subset of its edges: $E' \subseteq E$.

- Hence, a spanning tree of a graph $G = (V, E)$ is tree $T = (V, E')$ with the same node set V but E' are the edges of a tree.
- A spanning k-tree of $G = (V, E)$ is a k tree over V that can be embedded into G.

Spanning Sub-graphs

Definition 3.4.4 (Spanning Sub-graph)

A graph $G' = (V, E')$ is a spanning sub-graph of another graph $G = (V, E)$ if $E' \subseteq E$. I.e., G' is a spanning subgraph of G if G' can be embedded into G, specifically G' has the same node set as G and a subset of its edges: $E' \subseteq E$.

- Hence, a spanning tree of a graph $G = (V, E)$ is tree $T = (V, E')$ with the same node set V but E' are the edges of a tree.
- A spanning k-tree of $G = (V, E)$ is a k tree over V that can be embedded into G.
- Why the word "spanning"? It comes from linear algebra and matroid theory, where a set is said to span a space if it reaches or covers all elements – a spanning tree similarly connects all nodes of the graph.

Partial k-trees

Definition 3.4.5 (partial k-tree)

Any spanning sub-graph of a k-tree is a partial k-tree.

- Any partial k-tree is embeddable into a k-tree.

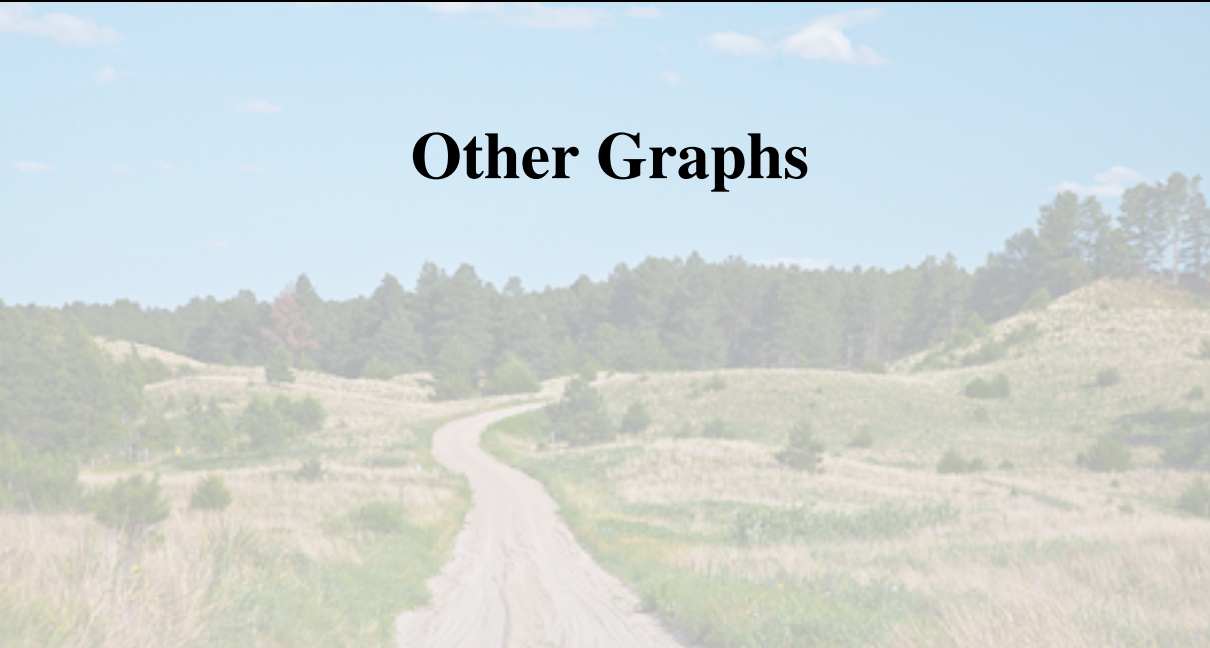
Partial k-trees

Definition 3.4.5 (partial k-tree)

Any spanning sub-graph of a k-tree is a partial k-tree.

- Any partial k-tree is embeddable into a k-tree.
- k trees and graphs that can be embedded into a k-tree for minimum k is important for probabilistic inference (as we will see in the upcoming weeks).

Other Graphs



Dynamic Graphs

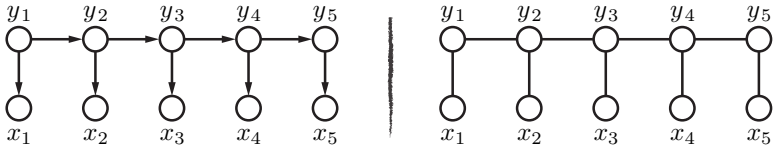
A dynamic graph is a graph whose structure (nodes, edges, or both) models and possibly evolves over time. One may think of a sequence of graphs $G_1, G_2, \ldots, G_T$, where each G_t has vertex set V_t and edge set E_t, potentially changing with t.

Examples:

- Hidden Markov Models (HMMs): A chain structure over time, with hidden states and observed symbols at each time step.
- Conditional Random Fields (CRFs): Often used for sequence labeling, can be seen as a dynamic graph unrolled over time with conditional dependencies.
- General Dynamic Graphical Models: Where edges and conditional dependencies can shift as new observations arrive.
- Multilayer and Multiplex networks: A graph consisting of multiple layers (like in a DNN or GNN) evolving over time or compute stages, or alternatively each layer represents different kinds of relationships between nodes.

Dynamic Graphs: Hidden Markov Model

- Nodes correspond to hidden states y_t and observations x_t.
- Edges link y_{t-1} to y_t (1st-order Markovian dependency) and y_t to x_t (observation model).
- The overall model unrolled over T time steps forms a chain (or chain-like) structure.



- An HMM is a tree (in the GM sense). Computing inference (over cliques) has cost $O(r^2)$, with T cliques, overall cost is $O(Tr^2)$.
- Also, we view $x_{1:T}$ as the stochastic process over time, underlying generative hidden chain $y_{1:T}$, giving family of distributions

$$p(x_{1:T}) = \sum_{y_{1:T}} p(x_{1:T}, y_{1:T}) \tag{3.12}$$

- $p(x_{1:T})$ computation looks exponential $O(r^T)$, but is not since tree graph (only $O(Tr^2)$).

Conditional Random Fields (CRFs)

A linear-chain CRF (henceforth CRF) is a conditional distribution over $2T$ random variables $X_{1:T}$ and $Y_{1:T}$ that that factorizes as follows:

$$p(y|x) = p(y_{1:T}|x_{1:T}) = \frac{1}{Z(x)} \prod_{t=1}^{T} g(x_t, y_t) \prod_{t=2}^{T} h(y_t, y_{t-1}) \tag{3.13}$$

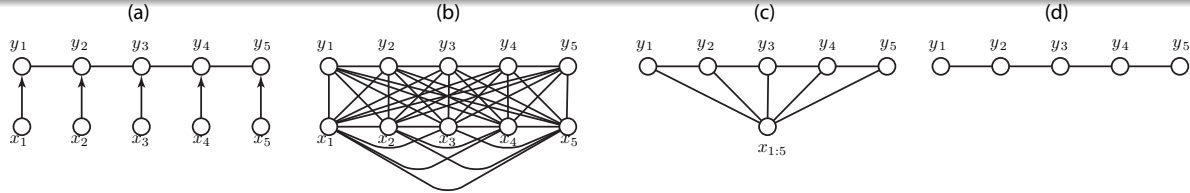
$$= \frac{1}{Z(x)} \prod_{t} h(y_t, y_{t-1}) g(x_t, y_t) \tag{3.14}$$

where

$$Z(x) = \sum_{y_{1:T}} \prod_{t} h(y_t, y_{t-1}) g(x_t, y_t) \tag{3.15}$$

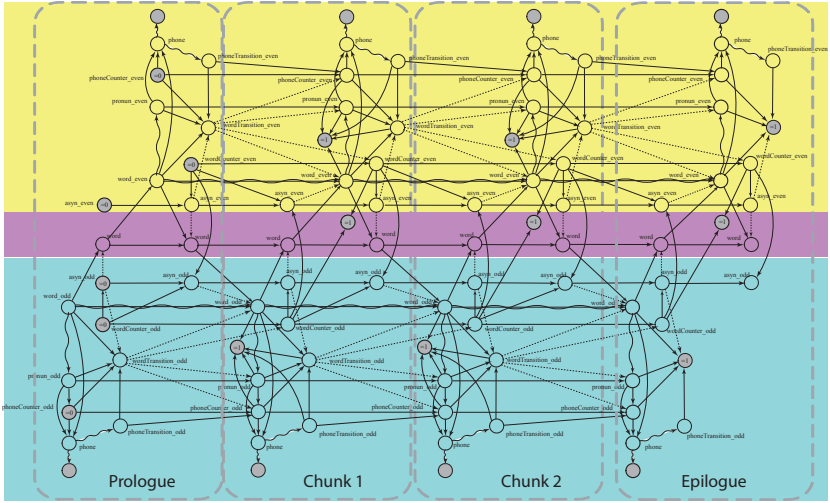
where $h(\cdot, \cdot)$ and $g(\cdot, \cdot)$ are arbitrary non-negative functions of their respective arguments.

CRF for Sequence Labeling



- Nodes often represent labels at each position of a sequence (like HMMs).
- Edges encode pairwise label dependencies (e.g., between neighboring time steps), plus edges from inputs if needed.
- Graph evolves step-by-step if we consider an online data stream.
- CRF corresponds to inherently conditional distribution $p(y|x)$ rather than an HMM which represents a generative distribution $p(x, y)$.
- In CRF, need only that the hidden variables y factorize, the input x is always observed, does not contribute to state space.
- Often convex, so relatively easy to optimize (many simple iterative approaches, e.g., perceptron updates or gradient steps).

Dynamic Graphical Model: Polyphase Speech Recognition

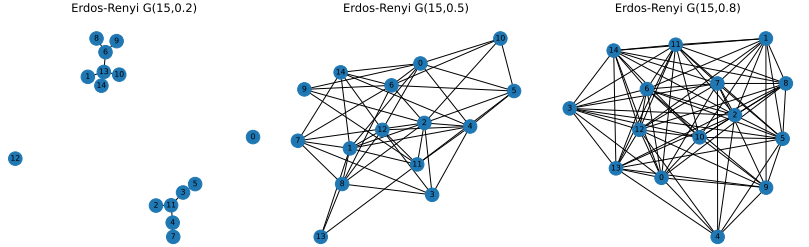


Random Graphs

A random graph is a graph generated by some probabilistic process. One of the simplest, classic, and most studied models is the Erdös-Rényi model.

Erdös-Rényi $G(n, p)$ Model:

- Start with n labeled nodes.
- For each pair of distinct nodes (i, j), an edge is included with probability p, independently of all other edges.
- The resulting graph can exhibit a range of properties (connectivity, presence of a giant component) depending on n and p.



Random Graphs: Examples and Properties

For example, $G(5, 0.5)$ might look like:

- 5 nodes, each pair connected with probability 0.5.
- On average, expected number of edges is $0.5 \times \binom{5}{2} = 5$.

Some properties that may be observed:

- Degree Distribution: Binomial or approximately Poisson (for large n and small p) degree distributions (when looking at a histogram of node degree of such graphs).
- Threshold Phenomena: For certain values of p, and larger n, emergence of a giant connected components can occur (to see this, need to sample a number of times).
- Phase Transitions: structural properties can appear abruptly at critical p values (e.g. graph connectivity).

Expander Graphs

- An expander graph is a sparse graph that has strong connectivity properties, specifically:

  For every vertex subset $S \subseteq V$, the neighborhood of S is large relative to $|S|$.

Expander Graphs

- An expander graph is a sparse graph that has strong connectivity properties, specifically:

  For every vertex subset $S \subseteq V$, the neighborhood of S is large relative to $|S|$.

- In other words, a not too large subset $S \subseteq V$ has a neighborhood at least a fraction $\alpha > 0$ as large as itself.

Expander Graphs

- An expander graph is a sparse graph that has strong connectivity properties, specifically:

  For every vertex subset $S \subseteq V$, the neighborhood of S is large relative to $|S|$.

- In other words, a not too large subset $S \subseteq V$ has a neighborhood at least a fraction $\alpha > 0$ as large as itself.
- Also, since this is true for every S, removing a small set of vertices (or edges) does not tend break the graph into small disconnected components.

Expander Graphs

- An expander graph is a sparse graph that has strong connectivity properties, specifically:

  For every vertex subset $S \subseteq V$, the neighborhood of S is large relative to $|S|$.

- In other words, a not too large subset $S \subseteq V$ has a neighborhood at least a fraction $\alpha > 0$ as large as itself.
- Also, since this is true for every S, removing a small set of vertices (or edges) does not tend break the graph into small disconnected components.
- Formal Criterion (Vertex Expansion): For $\alpha > 0$, a graph G is an α-vertex expander if for every $S \subseteq V$ with $|S| \leq \frac{n}{2}$, $|\Gamma(S)| \geq \alpha |S|$, where $\Gamma(S)$ denotes the set of neighbors of S (excluding S itself if we want an external boundary). That is

$$\Gamma(S) \triangleq \{v \in V \setminus S : \exists (u, v) \in E \text{ with } u \in S\} \tag{3.16}$$

  is the neighborhood function.

Expander Graphs: Properties and Examples

Properties:

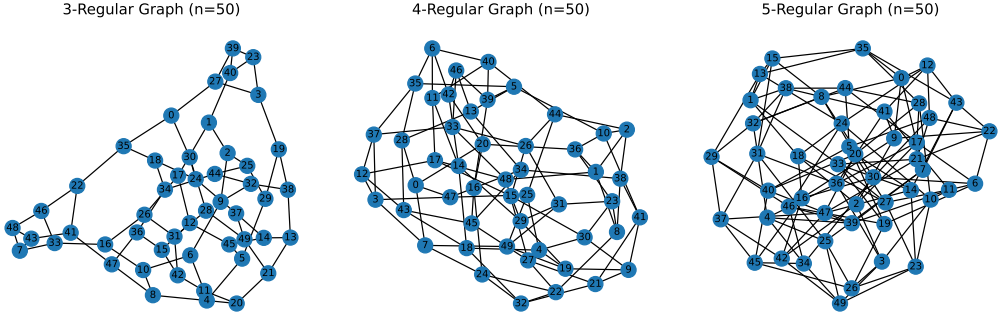
- High Connectivity: Even though the graph has relatively few edges (sparse), it remains robustly connected.
- Spectral Gap: We can show that expander graphs have a large gap between the largest and second-largest eigenvalues of their adjacency matrix (or Laplacian).
- Applications: Network design, error-correcting codes, pseudorandomness, etc.

Example: of an expander-like class of graphs.

- Definition: an undirected graph is regular if the node degree is constant, i.e., $d_v = d$ for some d and $\forall v \in V$.
- Definition: a d-regular graph has $d_v = d$ for the specific d for all v.
- A random d-regular graph is one selected randomly from all d-regular graphs on n nodes.
- Random d-regular graphs often exhibit expansion properties with high probability (next slide with some examples).

Example: Random d-regular Graphs

Random d-regular have high probabiity of being expanders.



These are easily generated using `networkx` via `nx.random_regular_graph(d, n)`.

Small-World Networks: Definition

A *small-world network* is one in which:

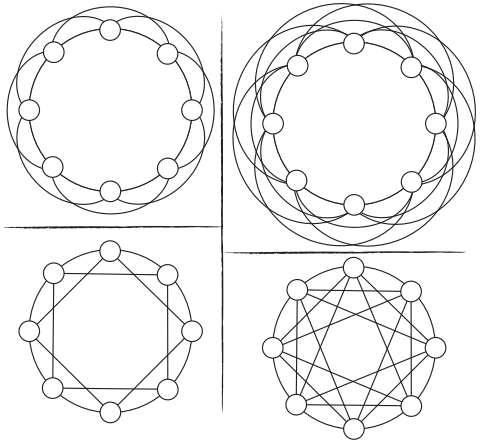
- Most nodes are not direct neighbors.
- The *average path length* between any two nodes is relatively small, growing at most logarithmically in the network size (so nothing is too far from anything else).
- Often includes a relatively high *clustering coefficient* C where $C =$ ratio of closed to closed+open triplets (open triplet: three nodes connected by two edges. closed triplet: three nodes connected by three edges, a triangle.)

Properties:

- Short Average Path Length: Even for large n, most nodes can be reached from any other node in a relatively small number of hops.
- High Clustering: Neighbors of a node tend to be neighbors with each other (like in social networks).
- Real-world examples: Social networks, some biological and technological networks (e.g. the web graph), "Six Degrees: The Science of a Connected Age", Duncan J. Watts, 2003.

Ring Lattices

A ring lattice is a graph that can be arranged in a circle with each node connected to the nearest neighbors along the circle.



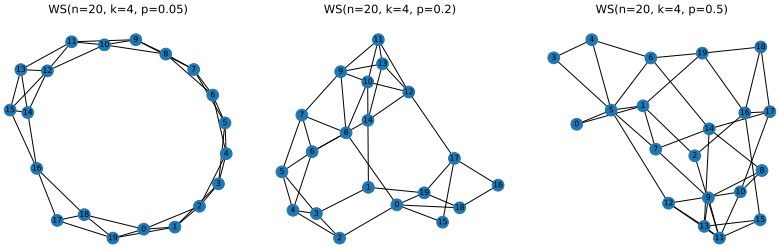
These are used as the base graph in Watts-Strogatz small-world network generation which we see next . . .

Example: Small World Watts-Strogatz Graphs

Example: The Watts-Strogatz model, where:

- Place n nodes uniformly around a circle.
- Connect each node to its k nearest neighbors, i.e., link each node to the $k/2$ closest nodes in the clockwise & counterclockwise directions (k is even), forming a ring lattice structure.
- For every edge, independently w.p. β, rewire one end to a new node chosen uniformly at random from entire graph. Random shortcuts.

Watts-Strogatz graph's tends to exhibit short average path length and high clustering coefficient, typical of small-world graphs (in the below p $= \beta$).



Generated using `networkx` using `nx.watts_strogatz_graph(n, k, p, seed=seed)`

Scale-Free Networks: Definition

A *scale-free network* is a network whose node degree distribution follows a power law:

$$P(\text{degree} = d) \propto d^{-\gamma}, \tag{3.17}$$

for some constant γ (often between 2 and 3). So a large number of nodes have small degree, and a small number of nodes have large degree.
That is:

- A small number of *hub* nodes have extremely high degree.
- Most nodes have comparatively few connections.
- The degree distribution is *heavy-tailed* or *long-tailed*. That is, a small fraction of nodes accounts for a large portion of the connections, while the majority of nodes have relatively few connections.

Scale-Free Networks: Examples and Generation

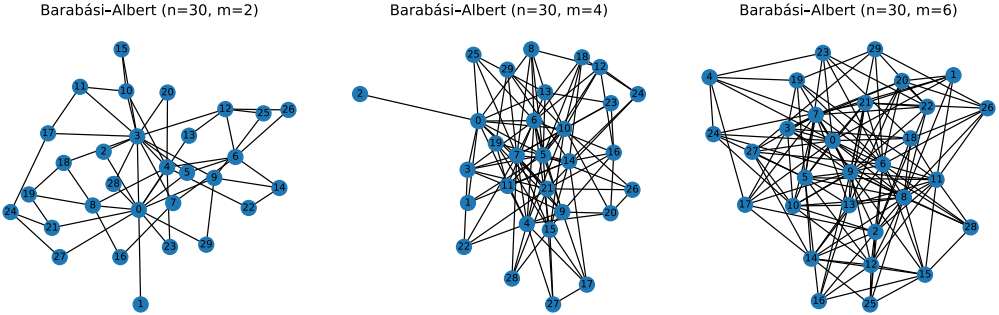
Examples:

- World Wide Web (few sites have extremely many links, most have few).
- Biological networks (metabolic, protein interaction) can be approximately scale-free.
- Social networks (celebrity hubs, many regular individuals).

Generating random scale-free networks:

- Preferential Attachment (Barabási–Albert model):
  1. Start with a small initial graph of m_0 nodes.
  2. Add new nodes one at a time, each new node creating edges to existing nodes with probability proportional to their current degrees.
- This yields a power-law degree distribution in the limit.

Example: Scale Free Graphs via Barabási–Albert model



Again, easily generated using `networkx` via `nx.barabasi_albert_graph(n, m, seed=seed)`

Recap

Dynamic Graphs: Sequences of graphs over time (e.g., HMMs, CRFs).

Random Graphs: Edges arise from a probability distribution (e.g., Erdös-Rényi).

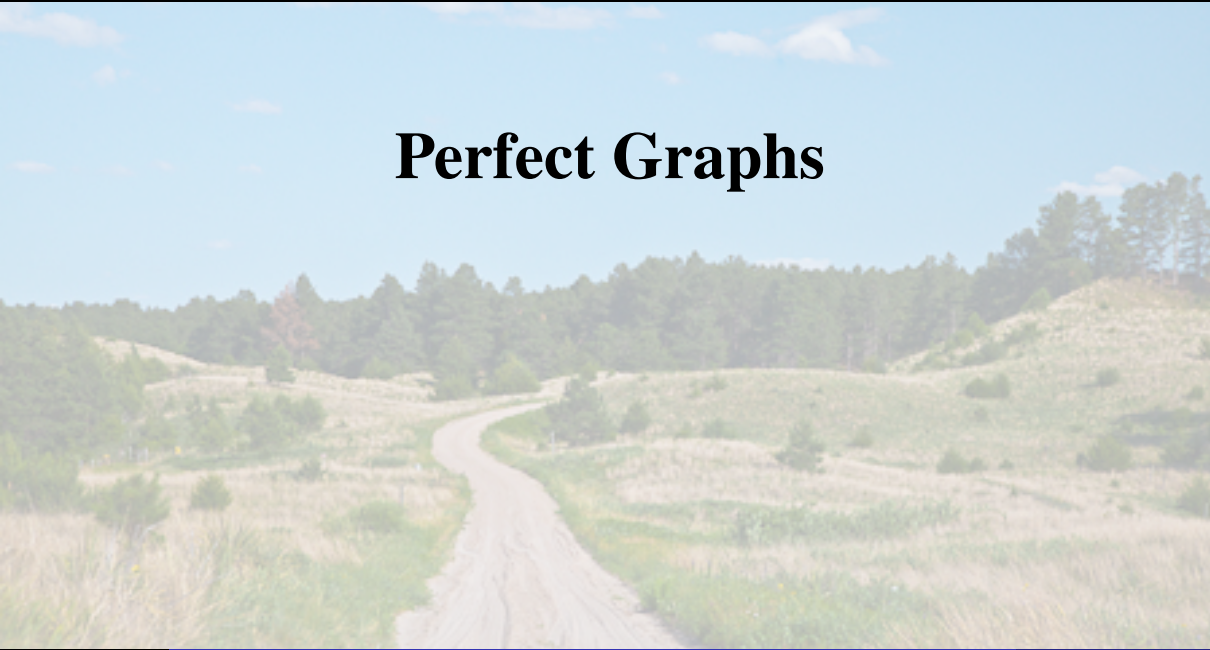
Expander Graphs: Sparse yet highly connected, robust to cuts (large vertex/edge expansion).

Small-World Networks: High clustering and small average path length (Watts–Strogatz).

Scale-Free Networks: Degree distribution follows a power law, featuring a few hubs and many low-degree nodes (Barabási–Albert).

Each category has distinct structural and probabilistic properties with numerous applications in network science, computer science, and beyond.

Perfect Graphs



Comparison of Graph Types

What's the difference between these graphs . . .

and these graphs?